# claude-windows / 50e27c80-8f59-4d8f-8a14-fdbddb9fbb3a

## Metadata

- Source: `claude-windows`
- Kind: `claude`
- Source file: `C:\Users\avidu\.claude\projects\C--Users-avidu-Projects-badminton-highlight-indexer\50e27c80-8f59-4d8f-8a14-fdbddb9fbb3a\subagents\workflows\wf_ddb0b3fd-b40\agent-a83545de0f7ddd069.jsonl`
- SHA-256: `a409f8e1b459265b3627a6048d06a208492b06d0be4d2a32c86c89ca43ebac76`
- Source modified: `2026-07-07T04:16:05+00:00`
- Imported at: `2026-07-08T16:00:26+00:00`
- project: `wf_ddb0b3fd-b40`
- session_id: `50e27c80-8f59-4d8f-8a14-fdbddb9fbb3a`

## Transcript

### 1. user (2026-07-07T04:14:53.574Z)

```
You are reviewing an UNCOMMITTED working diff implementing issue #506 (GPU-resident WASB
inference) in the repo at C:/Users/avidu/Projects/khelsutra-guru/badminton-highlight-indexer.
The full diff is at C:/Users/avidu/AppData/Local/Temp/claude/C--Users-avidu-Projects-badminton-
highlight-indexer/50e27c80-8f59-4d8f-8a14-fdbddb9fbb3a/scratchpad/506_diff.patch (read it). The
new test file tests/test_wasb_nvdec_resident.py is untracked (not in the diff) ? read it
directly from the repo.

Ground truth you must compare against:
- The VALIDATED reference implementation: C:/Users/avidu/Projects/khelsutra-guru/badminton-
highlight-indexer/scratch/gpu_resident/exp_t2_full.py (the GpuFrontend class + its grid build
ARE what was validated on a real L4: F1 0.574 ? baseline 0.582 on the hard clip). The production
port must be semantically faithful to it.
- WASB-SBDT interfaces (not in this repo; trust these facts):
TracknetV2Detector.run_tensor(imgs, affine_mats) moves imgs .to('cuda') itself and calls
postprocessor.run(preds, affine_mats) which does affine_mats[scale].cpu().numpy() then indexes
[i] per batch item; ImageDataset (eval) returns (imgs_t, hms_t, trans_outputs_inv, xys, visis,
img_paths) where trans_outputs_inv is {scale: np.float64 [2,3]}; per-frame preprocessing is PIL
RGB -> cv2.warpAffine(trans_input, (512,288), INTER_LINEAR) -> ToTensor -> Normalize(ImageNet);
get_transform uses c=(w/2,h/2), s=max(h,w); OnlineTracker uses np.Inf; WASB detector.py calls
torch.load(model_path) with no weights_only arg.
- The CPU decode path (run(), run_video_streaming) must be BEHAVIOR-UNCHANGED on the WSL
torch-1.11 + numpy<1.24 env (byte-parity guarantee), except that torch>=2.6 envs get the
weights_only + np.Inf shims.

Report ONLY defects you can ground in the actual code you read ? cite file:line. No style nits,
no speculative maybes without a concrete failure scenario. Return findings as structured output.

A reviewer claims this defect in the #506 diff:
- file: deploy/cloudrun/Dockerfile
- line: 81
- summary: PyNvVideoCodec is installed unpinned (`pip install -q PyNvVideoCodec`) in both deploy
paths (also deploy/digitalocean/gpu_setup.sh:71), unlike the rest of the '#506 validated combo'
which is explicitly pinned (torch==2.6.0, torchvision==0.21.0) ? and unlike numpy, whose
unpinning gets a justifying comment. The nvdec_resident frontend depends on version-sensitive
PyNvVideoCodec API surface (SimpleDecoder(..., output_color_type=nvc.OutputColorType.RGB),
DLPack frame layout), so a rebuild of the 'same' image can silently pick up an incompatible
future release.
- failure scenario: Months later the image is rebuilt (e.g. for an unrelated fix); pip resolves
a new PyNvVideoCodec major version whose SimpleDecoder/OutputColorType API changed; the build
succeeds and cpu-decode runs fine, but every decode_backend=nvdec_resident job on the paid GPU
fails at NvdecResidentFrontend construction ? the validated combo was never actually
reproducible from the Dockerfile.
- severity: minor
```

Adversarially VERIFY it: read the actual code (and the validated reference / WASB interface facts above) and try to REFUTE the claim. It is only real if the concrete failure scenario actually happens in the code as written. Default to isReal=false if the scenario cannot occur, is already guarded, or rests on a misreading.

### 2. user (2026-07-07T04:15:03.722Z)

```
10:+    # onto the GPU with no host round-trip: PyNvVideoCodec NVDEC decode ? affine
`grid_sample`
15:+    # runner only ? needs the torch-2.x + PyNvVideoCodec WASB env
(deploy/cloudrun/Dockerfile)
102:+            # (PyNvVideoCodec NVDEC) ? no PNG extraction and no cv2 streaming, so neither
139:+onto the GPU with no host round-trip: PyNvVideoCodec NVDEC decode ? affine
143:+PyNvVideoCodec env (the WSL torch-1.11 env keeps ``cpu``, its behaviour untouched).
153:+# resize + normalize (torch 2.x + PyNvVideoCodec, CUDA-only).
380:+    PyNvVideoCodec ``SimpleDecoder(output_color_type=RGB)`` decodes on the GPU's
388:+        import PyNvVideoCodec as nvc
400:+                f"PyNvVideoCodec RGB SimpleDecoder failed for {video_path}: {e!r} ? "
401:+                f"decode_backend=nvdec_resident needs PyNvVideoCodec with RGB output "
643:+        "(#506): PyNvVideoCodec NVDEC + cv2-faithful affine grid_sample, CUDA-only, "
691:+#   * a conda `wasb` env (py3.10 + torch 2.6.0+cu124 + PyNvVideoCodec ? the #506 modernized
stack,
701:+# the video-codec libs ? no libnvcuvid.so.1, no libnvidia-encode.so.1 ? and PyNvVideoCodec
needs
726:+# PyNvVideoCodec dlopens. Harmless when decode_backend=cpu; required for nvdec_resident.
743:+# PyNvVideoCodec (MIT ? the practical NVDEC path; TorchCodec needs a matching ffmpeg build
and could
754:     && "$CONDA_DIR/envs/wasb/bin/python" -m pip install -q \
756:-    && ( [ -f "$WASB_REPO_DIR/requirements.txt" ] && "$CONDA_DIR/envs/wasb/bin/python" -m
pip install -q -r "$WASB_REPO_DIR/requirements.txt" || true ) \
758:+    && "$CONDA_DIR/envs/wasb/bin/python" -m pip install -q PyNvVideoCodec \
759:     && "$CONDA_DIR/envs/wasb/bin/python" -m pip install -q \
780:+# PyNvVideoCodec for the GPU-resident decode backend (indexing.wasb.decode_backend=
782:+# PyNvVideoCodec needs (compute-only driver images ship neither).
799:+echo "[gpu] [2/6] NVDEC/NVENC driver userspace libs (#506: PyNvVideoCodec needs BOTH)"
833:+echo "[gpu] [4/6] conda env 'wasb' (py3.10 + torch 2.6.0+cu124 + PyNvVideoCodec ? #506
stack)"
836:-python -m pip install -q torch==1.11.0+cu113 torchvision==0.12.0+cu113 \
838:-[ -f "$WASB_REPO/requirements.txt" ] && python -m pip install -q -r
"$WASB_REPO/requirements.txt" || true
842:-python -m pip install -q hydra-core omegaconf pandas opencv-python-headless pillow scipy
tqdm pyyaml matplotlib "numpy<1.24" || true
843:+python -m pip install -q torch==2.6.0 torchvision==0.21.0 \
845:+python -m pip install -q PyNvVideoCodec
849:+python -m pip install -q hydra-core omegaconf pandas opencv-python-headless pillow scipy
tqdm pyyaml matplotlib numpy || true
869:+    import PyNvVideoCodec as pnvc
870:+    print("[gpu]       PyNvVideoCodec", getattr(pnvc, "__version__", "?"), "OK")
872:+    print("[gpu]       !! PyNvVideoCodec import failed:", repr(e)[:150])
887:+> | device NVIDIA L4` (plus a `PyNvVideoCodec ... OK` line ? the #506 GPU-resident decode
dep) and
901:+Ship the engine as **ONE pip-installable artifact whose behaviour is selected entirely by
the already-merged decoupled-compute seam** (`deployment.profile` / `compute_target` /
Storage+Compute backends), fronted by a friendly launcher + a Windows installer ? **not** a
frozen monolith, **not** an Electron/Tauri shell. The hard constraint that forces scope: torch
is deliberately out of `pyproject` (needs `--index-url`) **and** native WASB needs a *second,
isolated* torch env (conda py3.10 + torch 2.6+cu124 + PyNvVideoCodec since #506) shelled out via
`$WASB_PYTHON` `[verified native_wasb_runner.py:89]` ? so no single venv/freeze can deliver the
GPU-quality path. **The honest v1 is a LIGHT tier** (Gemini + motion + cpu-mock, *zero*
torch/GPU) that preserves CUJ-1..5/8; the native-WASB GPU path (CUJ-6/7) ships **owner-only**
until two blockers clear (WASB-C3 weight provenance + a reproducible truly-local config).
**LIGHT is the FIRST shippable increment, not the terminal scope ? the sole completion goal
(north star, L10) is the FULL quality path: native WASB + BYO training + in-app annotation.**
Delivery is **cross-platform (Windows/macOS/Linux): the same local webserver powers the UI on
all three, with NO native component** (L9). **Five small engine prerequisites must land before
```

any packaging push** (§7 P0). The single most actionable finding: **all mutable state is CWD-relative today and `--setup`/server disagree on `config.json`** ? a silent data-loss footgun that P0 must fix first.
923:+    # The #506 modernized WASB stack (L4-validated): torch 2.6 cu124 + PyNvVideoCodec, and
924:+    # the NVDEC/NVENC driver userspace libs PyNvVideoCodec dlopens (compute-only GPU hosts
928:+    assert "PyNvVideoCodec" in text

### 3. user (2026-07-07T04:15:04.763Z)

```
deploy\cloudrun\Dockerfile:11:#   * a conda `wasb` env (py3.10 + torch 2.6.0+cu124 +
PyNvVideoCodec ? the #506 modernized stack,
deploy\cloudrun\Dockerfile:21:# the video-codec libs ? no libnvcuvid.so.1, no libnvidia-
encode.so.1 ? and PyNvVideoCodec needs
deploy\cloudrun\Dockerfile:53:# PyNvVideoCodec dlopens. Harmless when decode_backend=cpu;
required for nvdec_resident.
deploy\cloudrun\Dockerfile:69:# PyNvVideoCodec (MIT ? the practical NVDEC path; TorchCodec needs
a matching ffmpeg build and could
deploy\cloudrun\Dockerfile:81:    && "$CONDA_DIR/envs/wasb/bin/python" -m pip install -q
PyNvVideoCodec \
deploy\digitalocean\gpu_setup.sh:11:# PyNvVideoCodec for the GPU-resident decode backend
(indexing.wasb.decode_backend=
deploy\digitalocean\gpu_setup.sh:13:# PyNvVideoCodec needs (compute-only driver images ship
neither).
deploy\digitalocean\gpu_setup.sh:35:echo "[gpu] [2/6] NVDEC/NVENC driver userspace libs (#506:
PyNvVideoCodec needs BOTH)"
deploy\digitalocean\gpu_setup.sh:66:echo "[gpu] [4/6] conda env 'wasb' (py3.10 + torch
2.6.0+cu124 + PyNvVideoCodec ? #506 stack)"
deploy\digitalocean\gpu_setup.sh:71:python -m pip install -q PyNvVideoCodec
deploy\digitalocean\gpu_setup.sh:93:    import PyNvVideoCodec as pnvc
deploy\digitalocean\gpu_setup.sh:94:    print("[gpu]       PyNvVideoCodec", getattr(pnvc,
"__version__", "?"), "OK")
deploy\digitalocean\gpu_setup.sh:96:    print("[gpu]       !! PyNvVideoCodec import failed:",
repr(e)[:150])
tests\test_compute_backend.py:693:    # The #506 modernized WASB stack (L4-validated): torch 2.6
cu124 + PyNvVideoCodec, and
tests\test_compute_backend.py:694:    # the NVDEC/NVENC driver userspace libs PyNvVideoCodec
dlopens (compute-only GPU hosts
tests\test_compute_backend.py:698:    assert "PyNvVideoCodec" in text
backend\config\models.py:123:    # onto the GPU with no host round-trip: PyNvVideoCodec NVDEC
decode ? affine `grid_sample`
backend\config\models.py:128:    # runner only ? needs the torch-2.x + PyNvVideoCodec WASB env
(deploy/cloudrun/Dockerfile)
docs\PACKAGING_PLAN.md:23:[Omitted long matching line]
backend\pipeline\detectors\wasb_infer.py:38:onto the GPU with no host round-trip: PyNvVideoCodec
NVDEC decode ? affine
backend\pipeline\detectors\wasb_infer.py:42:PyNvVideoCodec env (the WSL torch-1.11 env keeps
``cpu``, its behaviour untouched).
backend\pipeline\detectors\wasb_infer.py:67:# resize + normalize (torch 2.x + PyNvVideoCodec,
CUDA-only).
backend\pipeline\detectors\wasb_infer.py:983:    PyNvVideoCodec
``SimpleDecoder(output_color_type=RGB)`` decodes on the GPU's
backend\pipeline\detectors\wasb_infer.py:991:        import PyNvVideoCodec as nvc
backend\pipeline\detectors\wasb_infer.py:1003:                f"PyNvVideoCodec RGB SimpleDecoder
failed for {video_path}: {e!r} ? "
backend\pipeline\detectors\wasb_infer.py:1004:                f"decode_backend=nvdec_resident
needs PyNvVideoCodec with RGB output "
backend\pipeline\detectors\wasb_infer.py:1254:        "(#506): PyNvVideoCodec NVDEC +
cv2-faithful affine grid_sample, CUDA-only, "
backend\pipeline\detectors\native_wasb_runner.py:290:            # (PyNvVideoCodec NVDEC) ? no
PNG extraction and no cv2 streaming, so neither
docs\CLOUD_GPU_DRYRUN_GUIDE.md:97:> | device NVIDIA L4` (plus a `PyNvVideoCodec ... OK` line ?
the #506 GPU-resident decode dep) and
docs\COMPUTE_DECOUPLED_SERVING\runlogs\DEPLOY_REPORT_cloud-serving_alpha-
all1v1_2026-07-06.md:84:[Omitted long matching line]
tests\test_wasb_nvdec_resident.py:14:is a HARDWARE test ? it needs a CUDA GPU, PyNvVideoCodec,
```

the WASB repo, weights and
tests\test_wasb_nvdec_resident.py:256:# (inside the torch-2.x wasb env, which has torch+cuda, PyNvVideoCodec, cv2):
tests\test_wasb_nvdec_resident.py:287:    pytest.importorskip("PyNvVideoCodec")

### 4. user (2026-07-07T04:15:38.838Z)

```
# --- WASB detector env (micromamba + conda-forge ? BSD-licensed, NO Anaconda ToS; P3a, ADR-005) ---
# Enforces ADR-005 (every dependency ? code, data, weights, AND the build toolchain ? stays
# MIT/Apache/BSD): micromamba (BSD-3) + conda-forge (community) replaces Miniconda's Anaconda-ToS-gated
# `defaults` channels (the ?200-employee Terms-of-Service trap flagged in PACKAGING_PLAN.md §3).
# MAMBA_ROOT_PREFIX=$CONDA_DIR (=/opt/conda) keeps the env at $CONDA_DIR/envs/wasb so $WASB_PYTHON below
# is UNCHANGED.
#
# #506 modernized stack (validated end-to-end on a real L4, 2026-07-06): py3.10 + torch 2.6.0+cu124
# (torch ?2 needs py?3.9; WASB HRNet ports clean ? cpu F1 0.576 ? the torch-1.11 baseline 0.582) +
# PyNvVideoCodec (MIT ? the practical NVDEC path; TorchCodec needs a matching ffmpeg build and could
# not load) for decode_backend=nvdec_resident. wasb_infer.py carries the two version shims this bump
# needs (torch.load weights_only=False; the NumPy-2.0 np.Inf alias for WASB's tracker). numpy is
# UNPINNED (the old "numpy<1.24" was the torch-1.11 ABI pin). WASB-SBDT ships no requirements.txt ?
# the explicit list below is the (L4-validated) dependency closure its src/ imports need.
ENV MAMBA_ROOT_PREFIX=/opt/conda
RUN curl -sSL https://micro.mamba.pm/api/micromamba/linux-64/latest \
        | tar -xj -C /usr/local bin/micromamba \
    && micromamba create -y -q -n wasb -c conda-forge python=3.10 \
    && git clone --depth 1 https://github.com/nttcom/WASB-SBDT.git "$WASB_REPO_DIR" \
    && "$CONDA_DIR/envs/wasb/bin/python" -m pip install -q \
        torch==2.6.0 torchvision==0.21.0 --index-url https://download.pytorch.org/whl/cu124 \
    && "$CONDA_DIR/envs/wasb/bin/python" -m pip install -q PyNvVideoCodec \
    && "$CONDA_DIR/envs/wasb/bin/python" -m pip install -q \
        hydra-core omegaconf pandas opencv-python-headless pillow scipy tqdm pyyaml matplotlib numpy

# --- Main application env ---
WORKDIR /app
COPY pyproject.toml README.md ./
COPY requirements-trackers.txt ./
COPY backend ./backend
COPY training ./training
# Install with the [storage-gcs,cloud-run,auth] extras. storage-gcs: the cloud worker PULLS the gs://
# input and PUSHES the gs:// outputs/done-marker through backend.storage (google-cloud-storage is
# lazy-imported there); without it the job dies at the first gs:// fetch with `ModuleNotFoundError:
# google.cloud`. cloud-run: this SAME image is reused as the deployed CONTROL PLANE, which dispatches
# the L4 worker Job via google.cloud.run_v2 (backend/compute/cloud_run.py, lazy-imported); without it
# the control plane fails every dispatch with "google-cloud-run is required for CloudRunCompute's real
# client". Both are real Cloud Run failures the mock-client tests can't see (the SDKs
```

```
are faked in CI).
# auth: PyJWT[crypto] for M9 edge-auth (security.edge_auth_enabled ? the Cf-Access JWT verify the
# reused-as-control-plane deployment runs behind api.khelsutra.guru); jwt is lazy-imported and
# edge-auth is default-OFF, so the worker path is unaffected ? but without it a control-plane
# deploy with edge-auth ON needed a hand-built derived image (the A1 deploy's PyJWT overlay).
# D13/P12: `trackers` (ByteTrack's new home) is a git-only dep (broken PyPI wheel ?
# roboflow/trackers#474) so it's NOT pulled by `pip install .`; install it out-of-band before
# the main env, or the person-cue path crashes with ModuleNotFoundError. The immutable git SHA is
```

### 5. user (2026-07-07T04:15:39.843Z)

```
# Modern conda BLOCKS non-interactive `conda create` until the defaults-channel Terms of Service
# are accepted (CondaToSNonInteractiveError). Accept them so the env build doesn't fail on a fresh
# box. (Observed on a GCP Deep-Learning-VM image, 2026-06-20.)
conda tos accept --override-channels --channel https://repo.anaconda.com/pkgs/main >/dev/null 2>&1 || true
conda tos accept --override-channels --channel https://repo.anaconda.com/pkgs/r >/dev/null 2>&1 || true

echo "[gpu] [3/6] WASB-SBDT repo"
if [ ! -d "$WASB_REPO/.git" ]; then
  git clone --depth 1 https://github.com/nttcom/WASB-SBDT.git "$WASB_REPO"
fi

echo "[gpu] [4/6] conda env 'wasb' (py3.10 + torch 2.6.0+cu124 + PyNvVideoCodec ? #506 stack)"
conda env list | grep -q '/envs/wasb$' || conda create -y -q -n wasb python=3.10
conda activate wasb
python -m pip install -q torch==2.6.0 torchvision==0.21.0 \
  --index-url https://download.pytorch.org/whl/cu124
python -m pip install -q PyNvVideoCodec
# WASB-SBDT ships NO requirements.txt: src/ imports hydra, pandas, etc. ? this explicit list is
# the L4-validated dependency closure (#506). numpy is UNPINNED (the old "numpy<1.24" was the
# torch-1.11 ABI pin); wasb_infer.py shims the NumPy-2.0 np.Inf removal for WASB's tracker.
python -m pip install -q hydra-core omegaconf pandas opencv-python-headless pillow scipy tqdm pyyaml matplotlib numpy || true

echo "[gpu] [5/6] pretrained badminton weights"
WEIGHTS="$WASB_REPO/pretrained_weights/wasb_badminton_best.pth.tar"
if [ -f "$WEIGHTS" ]; then
  echo "[gpu]       found $WEIGHTS"
else
  echo "[gpu]       !! MISSING $WEIGHTS ? fetch wasb_badminton_best.pth.tar (Drive/bucket) before extraction."
fi

echo "[gpu] [6/6] GPU sanity check (in the wasb env)"
python - <<'PY'
import torch
ok = torch.cuda.is_available()
print("[gpu]       torch", torch.__version__, "| cuda_available", ok,
      "| device", (torch.cuda.get_device_name(0) if ok else "CPU"))
assert ok, "CUDA not available in the wasb env ? check the GPU driver / base image"
try:
```

```
    import PyNvVideoCodec as pnvc
    print("[gpu]       PyNvVideoCodec", getattr(pnvc, "__version__", "?"), "OK")
except Exception as e:  # nvdec_resident unavailable; the cpu decode path still works
    print("[gpu]       !! PyNvVideoCodec import failed:", repr(e)[:150])
PY

echo "[gpu] native WASB env ready.  Activate with:  source $CONDA_DIR/etc/profile.d/conda.sh && conda activate wasb"
echo "[gpu] (the LinuxNativeRunner ? backend/pipeline/detectors ? drives extraction once it lands; M3.5.)"

```

### 6. user (2026-07-07T04:16:05.278Z)

Structured output provided successfully